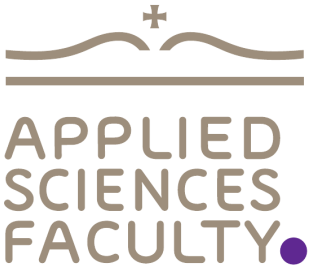
C++ — додаток. MySQL C API





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

MySQL

- MySQL — система керування реляційними базами даних. Open Source, друга після SQLite за поширенням.
- Детальніше див. `http://en.wikipedia.org/wiki/MySQL` і посилання там.

Інсталяція (DRAFT!)

- Linux/FReeBSD — можна заінсталювати, користуючись відповідним менеджером пакетів. Пам'ятайте, що для розробки потрібні також "-dev" пакети, котрі містять файли заголовків та імпорту!
- Для Windows простіше скористатися наперед скомпільованим. Взяти його можна тут[1]: `http://sourceforge.net/projects/mysql.mirror/files/`. Слід вибрати потрібну версію (остання на момент написання — MySQL 5.6.22), вибрати файл для вашої версії Windows та компілятора[2], наприклад mysql-5.6.22-win32.zip або mysql-installer-community-5.6.22.0.msi.
- Ще один варіант — скачати цілий пакет XAMPP, дистрибутив XAMPP: Apache, MySQL, PHP, Perl. Він, на додачу до повноцінного веб-сервера та купи інших корисних речей, містить і MySQL і phpMyAdmin — доволі зручний засіб адміністрування баз MySQL. Адреса: `http://sourceforge.net/projects/xampp/`

[1] З того часу, як Oracl-у дістався Sun, разом із купою важливих Open Source продуктів, таких як MySQL, згаданий Oracl ставить граблі у їх використанні. Зокрема, щоб скачати MySQL, слід реєструватися. Тому й даю вище пряме посилання на Sourceforge, де те ж саме доступно безпосередньо.

[2] Мається на увазі, що навіть під 64-бітним Windows, багато хто користується 32-бітними компіляторами. Зокрема, це стосується стандартних поставок MinGW та Visual Studio Express, хоча в обох випадках відносно легко можна отримати і 64-бітний компілятор.

- У вашому середовищі розробки слід встановити шляхи до фалів заголовків та бібліотек імпорту MySQL, щоб компілятор міг їх знайти.
- Слід вказати бібліотеку, з якою компонувати програму. Зазвичай це libmysqlclient (див. також нижче).
- Виклад нижче базується (для конкретності) на роботі із MySQL 5.6. Для інших версій можуть бути певні відмінності.

Коннектори

- Способів під'єднатися до цієї БД є багато.
- Ми будемо говорити про MySQL C API, в термінах MySQL — коннектор, який дозволяє маніпулювати БД за допомогою функцій C.
- C API ділиться на дві частини, libmysqlclient – клієнтська бібліотека, що дозволяє під'єднуватися до сервера. libmysqld — реалізація сервера, що дозволяє вбудовувати MySQL-сервер у свої програми.

Файли заголовків

- Оголошення засобів MySQL знаходяться у відповідних файлах заголовків. Найчастіше використовується `mysql.h`.
- Під Windows може бути потреба також явно включити `winsock2.h`.

```
#ifdef __WIN32__
#include <winsock2.h>
#endif
#include <mysql.h>
```

- Функції

  const char *mysql_get_client_info(**void**);
  const char *mysql_get_server_info(MYSQL *mysql);

  повертають стрічку із версією клієнтської чи серверної бібліотеки. Під час компіляції ця ж інформація доступна через макроси MYSQL_VERSION_ID та MYSQL_SERVER_VERSION, відповідно.
- Структура **MYSQL** описує конкретне під'єднання до БД. Власне, тому вона передається mysql_get_server_info() — треба ж знати, про який сервер цікавляться інформацією.
- Екземпляри MYSQL копіювати не можна! Немає гарантії, що такі копії будуть придатними до використання, однак, так як це звичайна структура C, заборонити копіювання (як би зробили для класу C++) неможливо.
-

- Перш ніж використовувати структуру MYSQL, її слід заповнити певними значеннями:

  MYSQL *mysql_init(MYSQL *mysql);

- Якщо їй передати 0 (NULL в традиціях чистого C), нульовий вказівник — вона виділяє пам'ять, заповнює структуру даними та повертає вказівник на неї.
- Якщо ж передати вказівник на екземпляр, що вже існує, просто заповнює його даними та повертає його ж адресу.
- Якщо не може виділити пам'ять, повертає нульовий вказівник.
- Викликає `mysql_library_init()`, яка ініціалізує бібліотеку. Тому або `mysql_init()`, або `mysql_library_init()` слід викликати **до виклику будь-яких інших функцій MySQL**.

- **void** mysql_close(MYSQL *mysql)

  Закриває під'єднання, звільняє пам'ять, виділену `mysql_init()` чи `mysql_connect()`.

Обробка помилок

- Якщо сталася помилка в роботі функції, вона повідомляє про це, повертаючи те чи інше спеціалізоване значення.
- Детальніше про помилку останнього виклику MySQL, можна довідатися, за допомогою:

  unsigned int mysql_errno(MYSQL *mysql);
  const char *mysql_error(MYSQL *mysql);

- mysql_errno() повертає код помилки, 0 якщо виклик був успішний. (Деякі функції не встановлюють цей код в 0, якщо все пройшло успішно, тому не слід нехтувати аналізом повернених значень. Поміж таких "лінивих" функцій — mysql_fetch_row(). Загальне правило — якщо функція звертається до сервера, вона таки має занулити код помилки, якщо виклик успішний).
- mysql_error() повертає текст із описом помилки. (Якщо попередній виклик завершився успішно, стрічка може містити текст останньої помилки, що відбулася раніше, або бути порожньою стрічкою).
- Коди помилок клієнта — див. `errmsg.h`, серверні коди — `mysqld_error.h`.

Під'єднання до БД

- Під'єднання можна здійснити за допомогою[3]:

```
MYSQL *mysql_real_connect(
        MYSQL *mysql,
        const char *host,
        const char *user,
        const char *passwd,
        const char *db,
        unsigned int port,
        const char *unix_socket,
        unsigned long client_flag );
```

- `host` — адреса сервера. Якщо 0 або містить "localhost", під'єднуватиметься до локального сервера.
- `user` — користувач БД.
- `passwd` — пароль користувача,
- `db` — база даних, до якою під'єднуємося.
- `port` — якщо е 0, вказує порт для TCP/IP-з'єднання. (Тип під'єднання визначається host).
- `unix_socket` — якщо не нуль, вказує сокет або іменований канал (named pipe) для взаємодії із БД. (Тип під'єднання визначається host).

[3] Існує ряд інших схожих функцій, детальніше див. документацію, наприклад

- `client_flag` — може бути 0, дозволяє вказати додаткові прапорці, наприклад (список не вичерпний, див. документацію!): CLIENT_COMPRESS, CLIENT_IGNORE_SPACE, CLIENT_REMEMBER_OPTIONS, CLIENT_MULTI_RESULTS, CLIENT_MULTI_STATEMENTS.
- По замовчуванню, для user і passwd використовується кодування latin1, змінити можна за допомогою виклику:

```
mysql_options(mysql, MYSQL_SET_CHARSET_NAME,
              "charset_name");
```

- Взагалі, `mysql_options()` дозволяє змінювати багато які опції, див. документацію, наприклад: `http://dev.mysql.com/doc/refman/5.6/en/mysql-options.html`.
- mysql_real_connect() повертає переданий їй вказівник на MYSQL, якщо під'єднання успішне, 0 в іншому випадку.

Приклад

```
void checkForSqlError(MYSQL *con){ // Обробка помилок
    std :: cerr << "Error: " << mysql_error(con) << std::endl;
    mysql_close(con);
    exit (1);
}
        ..............................

    MYSQL *con = mysql_init(NULL);
    if (con == NULL){
        cout << "mysql_init() returned 0" << endl;
        exit (1);
    }

    if( !mysql_real_connect(con, "localhost", "student",
                    "user", "teaching_db_translit", 0, NULL, 0) )
    {
        checkForSqlError(con);
    }
      ..............      Тіло програми      ..................
    mysql_close(con);
```

SQL-запити — 1

- Виконати SQL-запит можна за допомогою:

  int mysql_query(MYSQL *mysql, **const char** *stmt_str);

  де stmt_str — власне, C-стрічка із запитом. Стрічка може не містити крапки з комою в кінці, якщо запит тільки один (щодо підтримки кількох запитів за раз – див. документацію, `http://dev.mysql.com/doc/refman/5.6/en/c-api-multiple-queries.html`).
- Повертає 0, якщо виконалася успішно.

SQL-запити — 2

- Так як запит для mysql_query є звичайною C-стрічкою, він не може містити двійкових даних. На такий випадок можна використати:

  int mysql_real_query(MYSQL *mysql, **const char** *stmt_str, **unsigned long** length)

  де stmt_str вказує на послідовність байт (може містити нулі!), а length задає довжину цієї послідовності.
- Якщо в SQL-запит потрібно передати стрічки із спеціальними символами SQL, можна використати функцію mysql_real_escape_string(), яка їх загорне у escape-послідовності, актуальні для даної кодової таблиці (тому на момент її виклику зв'язок із сервером має вже бути встановлений — кодові таблиці залежать від налаштувань сервера). Детальніше див. документацію, наприклад: `http://dev.mysql.com/doc/refman/5.6/en/mysql-real-escape-string.html`.

Приклад:

- ```
  if ( !mysql_query(con, "CREATE DATABASE students") )
  {
      checkForSqlError(con);
  }
  ```

Аналогічним чином можна виконувати будь-які запити SQL!
  ```

Отримання даних із БД — 1

- Запит SELECT[4] — особливий, він повертає дані, які слід якось в клієнтської частини MySQL забрати.
- Найпростіший спосіб — виклик функції:

  MYSQL_RES *mysql_store_result(MYSQL *mysql);

  Вона повертає структуру `MYSQL_RES`, що містить вибрані запитом рядки (в термінах MySQL — result set).
- Покористувавшись, результати слід звільнити:

  void mysql_free_result(MYSQL_RES *result);

- Для роботи із "багатозапитними" результатами, використовується mysql_next_result(), детальніше ми її не розглядатимемо.
- Повертає нуль, якщо результатів не було (для запитів, які їх не повертають, таких як INSERT), або відбулася якась помилка.
- Повертає **порожню MYSQL_RES**, якщо результат порожній, не містить рядків. Не слід плутати із нульовим вказівником із попереднього пункту!

[4] А також, всілякі SHOW, DESCRIBE, EXPLAIN, CHECK TABLE, і т.д.

Отримання даних із БД — 2

- **unsigned int** mysql_field_count(MYSQL *mysql);

  повертає кількість стовпців, що мала бути в результаті запиту. Дозволяє перевірити, чи mysql_store_result() повернула нуль, бо немає що повертати, чи сталася помилка. (Також, якщо запит був успішним, після mysql_store_result(), mysql_errno() повертатиме 0, а mysql_error() порожню стрічку).
- Подивитися, скільки рядків є в MYSQL_RES, можна за допомогою:

  my_ulonglong mysql_num_rows(MYSQL_RES *result);

  Повертає коректне значення для mysql_store_result(), для mysql_use_result() (див. далі) результат буде некоректним, поки не будуть отримані всі рядки запиту.

Отримання даних із БД — 3

- Для запитів, таких як INSERT, UPDATE, DELETE, REPLACE, тощо, кількість рядків, на які вони вплинули, можна довідатися за допомогою:

  my_ulonglong mysql_affected_rows(MYSQL *mysql);

  (Вважається, REPLACE модифікує два рядки – один знищує, один вставляє.)
- Повертає кількість модифікованих рядків, 0 якщо жоден не було модифіковано, також якщо жоден не задовольнив умову WHERE, або запитів взагалі поки не було. Повертає -1, якщо сталася помилка, або після запиту не було викликано mysql_store_result().
- **УВАГА**: так як результат є беззнаковим, перевіряти, чи він рівний -1, варто за допомогою `(my_ulonglong)-1` або `(my_ulonglong) 0`!
-

Отримання даних із БД — 4

- З тих чи інших причин (наприклад, економія пам'яті), mysql_store_result(), яка читає всі дані, може не підходити. Тоді слід скористатися:

  MYSQL_RES *mysql_use_result(MYSQL *mysql)

  яка ініціалізує MYSQL_RES, однак не заповнює жодними даними. Тоді для отримання даних слід використовувати mysql_fetch_row(). Дані отримуватимуться безпосередньо із сервера.
- Поки дані отримуються, відповідні таблиці заблоковано, тому використовувати слід обережно.
- Якщо не всі дані було забрано (про кінець даних свідчить NULL, повернений mysql_fetch_row()), залишок "дістанеться" наступному запиту, та деякі запити генеруватимуть помилки.
- Деталі див., наприклад, тут: `http://dev.mysql.com/doc/refman/5.6/en/mysql-use-result.html`.

Робота з MYSQL_RES — 1

- Існує ціла колекція функцій для роботи із MYSQL_RES.
- MYSQL_ROW mysql_fetch_row(MYSQL_RES *result);

  повертає наступний (ще не отриманий) рядок із запиту.
- MYSQL_ROW — структура, що зберігає один рядок даних.
- Повертає 0, якщо рядки закінчилися (або сталася помилка, у випадку використання mysql_use_result()).
- Кількість рядків також можна взнати функцією mysql_num_rows() (див. вище).

Робота з MYSQL_RES — 2

Навігація по результатах. **Увага**: функції навігації нижче можна використовувати тільки для повних наборів даних, тобто — тільки після mysql_store_result().

- **void** mysql_data_seek(MYSQL_RES *result, my_ulonglong offset);

  переходить до запиту із номером offset. Offset може бути між 0 та mysql_num_rows(result)-1.

- MYSQL_ROW_OFFSET mysql_row_seek(MYSQL_RES *result, MYSQL_ROW_OFFSET offset)

  Переміщення до вказаного рядка. **УВАГА**, тут offset — не номер, а величина типу `MYSQL_ROW_OFFSET`, результат роботи mysql_row_seek() або mysql_row_tell()! Повертає старе значення offset.

- Взнати поточне положення можна за допомогою:

  MYSQL_ROW_OFFSET mysql_row_tell(MYSQL_RES *result)

Робота з MYSQL_ROW — 1

- Фактично, містить масив C-стрічок.
- **unsigned int** mysql_num_fields(MYSQL_RES *result);

  повертає кількість полів (стовпців) у результаті. (Див. також mysql_field_count() вище).
- Один рядок, одна змінна типу MYSQL_ROW, являє собою, фактично, масив, розміром, що повертає mysql_num_fields(). Тобто, звернутися до результатів можна так: row[0] ... row[mysql_num_fields(result)-1].
- Для порожніх полів – тих, що містять NULL в сенсі SQL, структура MYSQL_ROW міститиме NULL-вказівник.
- **unsigned long*** mysql_fetch_lengths(MYSQL_RES *result);

  повертає масив розмірів окремих полів переданого рядка.
- Повертає 0 як для порожніх полів, так і для тих, що містять NULL (той що SQL NULL). (Тому, щоб відрізнити, слід дивитися на row[i]).

Приклад:

```
if ( mysql_query(con, "SELECT * FROM students")!=0 ){
    checkForSqlError(con);
}

MYSQL_RES *result = mysql_store_result(con);
if ( result == NULL){
    checkForSqlError(con);
}

int num_fields = mysql_num_fields(result);

MYSQL_ROW row;

while ( (row = mysql_fetch_row(result)) ){
    for(int i = 0; i < num_fields; i++){
        cout << ( row[i] ? row[i] : "NULL" ) << " ";
    }
    cout << endl;
}

mysql_free_result( result );
```

Додаткові засоби — 1

- MYSQL_FIELD *mysql_fetch_field(MYSQL_RES *result);

  повертає структуру, MYSQL_FIELD, що містить опис кожного наступного стовпця результатів запиту. (Не дані — лише метадані стовпця!) Повертає NULL, якщо дані закінчилися.
- Поля цієї структури див. документацію: `http://dev.mysql.com/doc/refman/5.6/en/c-api-data-structures.html`. (Обов'язково подивіться!)
- Наступна функція:

  my_ulonglong mysql_insert_id(MYSQL *mysql);

  повертає значення, встановлене для поля типу AUTO_INCREMENT попередньою операцією INSERT або UPDATE. Детальніше див. документацію, наприклад: `http://dev.mysql.com/doc/refman/5.6/en/mysql-insert-id.html`.

Додаткові засоби — 2

- `mysql_get_character_set_info()` — інформація про кодову таблицю.
- `mysql_get_host_info()` — опис під'єднання.
- `mysql_get_proto_info()` — версія протоколу під'єднання.
- `mysql_info()` — стрічка з інформацією про останній виконаний запит.
- `mysql_stat()` — статус сервера.
- `mysql_thread_safe()` — повертає 1, якщо клієнт скомпільовано як потокобезпечний, 0 – в іншому випадку.
- `mysql_warning_count()` — кількість попереджень (warnigns) попереднього SQL-запиту.
- `mysql_ping()` — перевірити, чи сервер під'єднано, перепід'єднується, якщо є потреба.
- `mysql_list_tables()` — повертає список таблиці згідно регулярного виразу, `mysql_list_dbs()` — аналогічно для баз, `mysql_list_fields()` — полів, `mysql_list_processes()` — процесів сервера.

Детальніше див. посилання тут: `http://dev.mysql.com/doc/refman/5.6/en/c-api-function-overview.html`.

- MySQL надає простеньку обгортку — набір класів, для використання з C++, MySQL Connector/C++.
- Більш просунутою альтернативою може бути, наприклад, MySQL++: `http://tangentsoft.net/mysql++/doc/` — "*Third-party connector, MySQL++ "MySQL++ is a C++ wrapper for MySQL's C API. It is built around the same principles as the Standard C++ Library, to make dealing with the database as easy as dealing with STL containers. In addition, MySQL++ provides facilities that let you avoid the most repetitive sorts of SQL within your own code, providing native C++ interfaces for these common tasks.*"

DRAFT

- Запуск в парі mingw/mysql трохи нетривіальний.
- Див. `http://dev.mysql.com/doc/connector-cpp/en/connector-cpp-getting-started-examples.html` та `http://sourceforge.net/projects/windiana.u/files/mysql/port-mingw/`.











Багатопоточні міркування — 1

- MySQL C API *майже* потокобезпечне. Лише функції роботи із сокетами не є потокобезпечними (файд sql/net_serv.cc). (Детальніше див. `http://dev.mysql.com/doc/refman/5.6/en/c-api-threaded-clients.html`.)
- Використання різних під'єднань (connections) є потокобезпечним.
- Однак, коли декілька потоків користуються одним і тим же під'єднанням, слід дотримуватися ряду правил. Між парою викликів `mysql_query()` та `mysql_store_result()` жоден інший потік не повинен "вклинюватися". (Різні результати, збережені `mysql_store_result()` можуть використовуватися одночасно.) Також, інші потоки не повинні працювати із під'єднанням, поки використовується `mysql_use_result()` і відповідна множина результатів не закрита.

Багатопоточні міркування — 2

- mysql_init() створює ряд змінних, пов'язаних із потоком та викликає mysql_library_init(), яка не є потокобезпечною. Тому, слід або викликати mysql_init() перш ніж створювати потоки, або захищати виклик mysql_library_init() мютексами. Детальніше див. тут `http://dev.mysql.com/doc/refman/5.6/en/mysql-library-init.html`.
- Взагалі, рекомендована схема роботи виглядає так:
  1. `mysql_library_init()` викликати до створення потоків.
  2. Перш ніж використовувати будь-які функції MySQL, в кожному потоці викликати `mysql_thread_init()`. (Якщо викликатиметься `mysql_init()`, для створення об'єкту MYSQL, вона це самостійно зробить.)
  3. Перед завершенням потоку слід викликати `mysql_thread_end()`.
- Перевірити, чи клієнтську бібліотеку було скомпільовано як потокобезпечну, можна використати функцію:

```
unsigned int mysql_thread_safe(void);
```

Посилання — 1

- `http://dev.mysql.com/doc/refman/5.6/en/index.html` — Manual для MySQL 5.6.
- Зокрема: `http://dev.mysql.com/doc/refman/5.6/en/c-api.html` – MySQL C API; `http://dev.mysql.com/doc/refman/5.6/en/c-api-data-structures.html` — C API Data Structures; `http://dev.mysql.com/doc/refman/5.6/en/c-api-multiple-queries.html` — виконання декількох запитів однією командою.
- `http://tangentsoft.net/mysql++/doc/` – сторонній C++ конектор: "*Third-party connector, MySQL++ "MySQL++ is a C++ wrapper for MySQL's C API. It is built around the same principles as the Standard C++ Library, to make dealing with the database as easy as dealing with STL containers. In addition, MySQL++ provides facilities that let you avoid the most repetitive sorts of SQL within your own code, providing native C++ interfaces for these common tasks.*"
- `http://zetcode.com/databases/mysqltutorial/` — простий підручник MySQL, зокрема: `http://zetcode.com/db/mysqlc/` — MySQL C API.

Посилання — 2

- `http://sourceforge.net/projects/mysql.mirror/` — вільно доступне дзеркало MySQL, включаючи скомпільовані варіанти для різних систем.
- `http://www.mysql.com/` — офіційний сайт.
- `http://en.wikipedia.org/wiki/MySQL` — сторінка вікіпедії про MySQL.
-

